ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА»
(СПбГУТ)

АРХАНГЕЛЬСКИЙ КОЛЛЕДЖ ТЕЛЕКОММУНИКАЦИЙ
ИМ. Б.Л. РОЗИНГА (ФИЛИАЛ) СПбГУТ
(АКТ (ф) СПбГУТ)

КУРСОВОЙ ПРОЕКТ

НА ТЕМУ

РАЗРАБОТКА ВЕБ-САЙТА

«УСЛУГИ ПО ВЕДЕНИЮ

БУХГАЛТЕРСКОГО УЧЁТА»

Л109. 25КП01. 021 ПЗ

(Обозначение документа)

МДК.02.01 Технология разработки

программного обеспечения

Студент	ИСПП-35		08.12.2025	А.А. Фадеева
	(Группа)	(Подпись)	(Дата)	(И.О. Фамилия)
Преподаватель			09.12.2025	Ю.С. Маломан
		(Подпись)	(Дата)	(И.О. Фамилия)

Архангельск 2025

СОДЕРЖАНИЕ

Перечень сокращений и обозначений 3
Введение 4
1 Анализ и разработка требований 6
1.1 Назначение и область применения 6
1.2 Постановка задачи 6
1.3 Выбор состава программных и технических средств 9
2 Проектирование программного обеспечения 10
2.1 Проектирование интерфейса пользователя 10
2.2 Разработка архитектуры программного обеспечения 10
2.3 Проектирование базы данных 12
3 Разработка и интеграция модулей программного обеспечения 13
3.1 Разработка программных модулей 13
3.2 Реализация интерфейса пользователя 15
3.3 Разграничение прав доступа пользователей 16
3.4 Экспорт и импорт данных 18
4 Тестирование и отладка программного обеспечения 20
4.1 Структурное тестирование 20
4.2 Функциональное тестирование 22
5 Инструкция по эксплуатации программного обеспечения 23
5.1 Установка программного обеспечения 23
5.2 Инструкция по работе 23
Заключение 26
Список использованных источников 27

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем курсовом проекте применяют следующие сокращения и обозначения.

БД – база данных

ОС – операционная система

ПО – программное обеспечение

СУБД – система управления базами данных

API – интерфейс программирования приложения

ASP.NET – активные страницы сервера для .NET

DTO – объект передачи данных

GUI – графический интерфейс пользователя

HTTP – протокол передачи гипертекста

IDE – интегрированная среда разработки

JSON – текстовый формат обмена данными

JWT – веб-токен JSON

MVC – модель-представление-контроллер

MSSQL – Microsoft SQL Server

ERD – Entity-Relationship Diagram

EF – Entity Framework

SSMS – SQL Server Management Studio

ВВЕДЕНИЕ

В современных компаниях работа с клиентами часто ведется через звонки или сообщения, из-за чего информация может теряться, а ответы на запросы задерживаются. Это затрудняет работу сотрудников и снижает точность ведения данных. Внедрение автоматизированной системы обработки обращений позволяет централизовать все запросы, отслеживать их статус в реальном времени и сокращать время реакции на них. Кроме того, такая система обеспечивает удобный анализ поступающих данных и планирование работы сотрудников, что повышает эффективность взаимодействия с клиентами.

Актуальность разрабатываемого проекта обусловлена необходимостью перенести прием и обработку заявок на бухгалтерские услуги в одну удобную систему.

Разработка веб-сайта «Financial Solutions. Услуги по ведению бухгалтерского учёта» позволит собрать все обращения в одном месте, быстрее отвечать клиентам и сделать работу компании понятной и прозрачной. Это поможет лучше контролировать задачи и повысить качество обслуживания.

Целью курсового проектирования является разработка веб-сайта для управления заявками, который позволит регистрировать обращения и следить за их выполнением в режиме онлайн. Это упростит работу сотрудникам, снизит риск ошибок и сделает общение между клиентом и компанией более удобным. Для достижения поставленной цели требуется решить следующие задачи:

- провести сбор требований целевой аудитории;
- проанализировать информационные источники по предметной области;
- спроектировать архитектуру веб-сайта;
- спроектировать диаграмму вариантов использования приложения;

- выбрать состав программных и технических средств для реализации приложения;
- спроектировать БД;
- спроектировать интерфейс веб-сайта;
- создать БД в выбранной СУБД;
- разработать API для некоторых функций приложения;
- реализовать разграничение прав доступа пользователей;
- разработать интерфейс веб-сайта;
- разработать веб-сайт;
- реализовать работу веб-сайта с сервером БД при помощи REST API;
- выполнить структурное тестирование ПО;
- выполнить функциональное тестирование ПО;
- разработать программную и эксплуатационную документацию.

В результате выполнения поставленных задач будет разработан веб-сайт, обеспечивающий эффективное управление заявками и автоматизацию обработки обращений клиентов.

1 Анализ и разработка требований

1.1 Назначение и область применения

Веб-сайт «Financial Solutions. Услуги по ведению бухгалтерского учёта» предназначен для организаций, оказывающих финансовые и бухгалтерские услуги. Система позволит упростить общение между клиентами и специалистами, а также ускорить приём и обработку заявок на сопровождение бизнеса. Бухгалтеры и администраторы смогут эффективно отслеживать статус обращений и управлять рабочими задачами в одном месте.

1.2 Постановка задачи

Необходимо разработать веб-сайт, который предоставит доступ к следующей функциональности:

- авторизации и регистрации пользователей;
- добавлению и редактированию заявок на услуги;
- просмотру и редактированию списка услуг;
- просмотру информации о заявках;
- формированию отчётов в формате *.xlsx и *.csv по выполненным и текущим заявкам.

К веб-сайту предъявляются следующие эксплуатационные требования:

- интуитивно понятный интерфейс пользователя;
- авторизация и разграничение прав доступа;
- устойчивость к сбоям и сохранение данных при ошибках сети.

Неавторизованный пользователь может просматривать страницу с услугами и регистрироваться.

Клиент может создавать и просматривать свои заявки, отслеживать статусы заявок, редактировать контактные данные.

Администратор имеет полный доступ ко всем заявкам, может изменять их статусы, добавлять и редактировать информацию об услугах.

На рисунке 1 изображена диаграмма вариантов использования приложения.

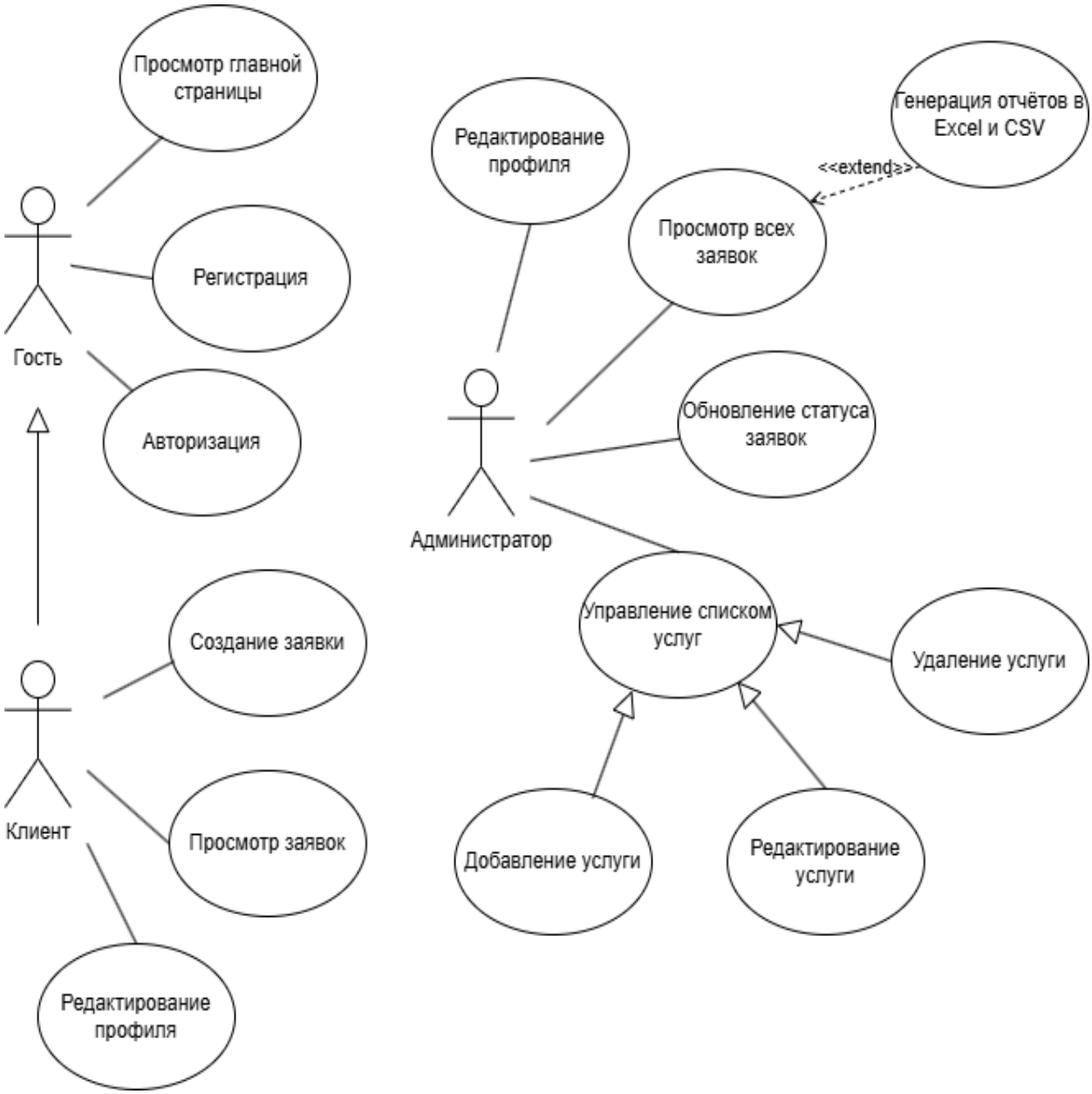

Рисунок 1 – Диаграмма вариантов использования

1.3 Выбор состава программных и технических средств

Согласно цели проекта требуется создать веб-сайт «Financial Solutions. Услуги по ведению бухгалтерского учёта» для автоматизации процессов приёма, хранения и обработки заявок.

Разработка веб-сайта будет осуществляться на языке C# с использованием фреймворка ASP.NET Core, обеспечивающего создание веб-сайтов с чёткой архитектурой и поддержкой MVC-паттерна.

В качестве СУБД выбрана MSSQL, отличающаяся стабильностью, простотой в развёртывании, а также наличием большого числа инструментов администрирования и интеграции с .NET-приложениями.

Для контейнеризации и упрощения развёртывания серверной части будет использоваться Docker, обеспечивающий переносимость и изоляцию сервисов. Конфигурация контейнеров будет управляться с помощью docker-compose.

Пользовательский интерфейс будет разработан при помощи ASP.NET Core.

Разработка будет выполняться в Visual Studio – IDE, обеспечивающей поддержку ASP.NET Core и работу с Docker и имеющей встроенные средства отладки.

Для функционирования системы на стороне сервера достаточны следующие программные и технические средства:

- ОС Ubuntu версии 18.04 и выше;
- сервер БД: MSSQL;
- процессор с 2 ядрами по 2 ГГц;
- оперативная память объемом 2 ГБ;
- ПО для конфигурирования, управления и администрирования сервера БД: SSMS последней версии;
- ПО для работы API: Docker, docker-compose, ASP.NET Core.

Для функционирования системы на стороне клиента достаточны следующие программные и технические средства:

- браузер с поддержкой HTML5 и CSS3;
- процессор с частотой 1 ГГц или быстрее;
- оперативная память в объеме 1 ГБ и выше;
- постоянное интернет-подключение.

2 Проектирование программного обеспечения

2.1 Проектирование интерфейса пользователя

В рамках разработки веб-сайта «Financial Solutions. Услуги по ведению бухгалтерского учёта» спроектирован интерфейс пользователя при помощи Figma. Wireframe позволяют увидеть основные элементы и функциональность приложения. Wireframe приложения показаны рисунке 2.

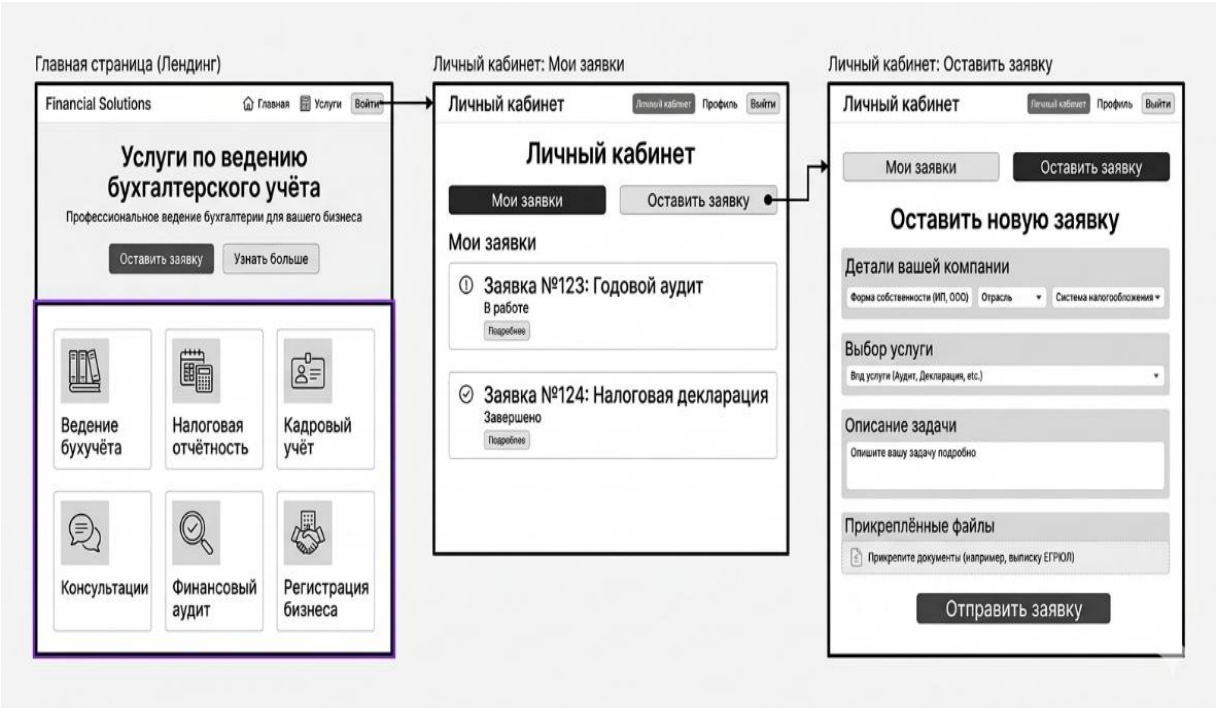

Рисунок 2 – Figma. Wireframe приложения

2.2 Разработка архитектуры программного обеспечения

Приложение предназначено для автоматизации процессов приёма, хранения и обработки заявок. Архитектура приложения построена на основе клиент-серверной модели и включает в себя следующие элементы: серверная часть приложения, веб-приложение, БД[3].

Для серверной части будет создан API, позволяющий приложению взаимодействовать с БД.

Приложение будет разработано с использованием ASP.NET Core. Взаимодействие будет осуществляться через HTTP-запросы к API. Ответы от сервера будут поступать в формате JSON. Диаграмма развёртывания представлена на рисунке 3

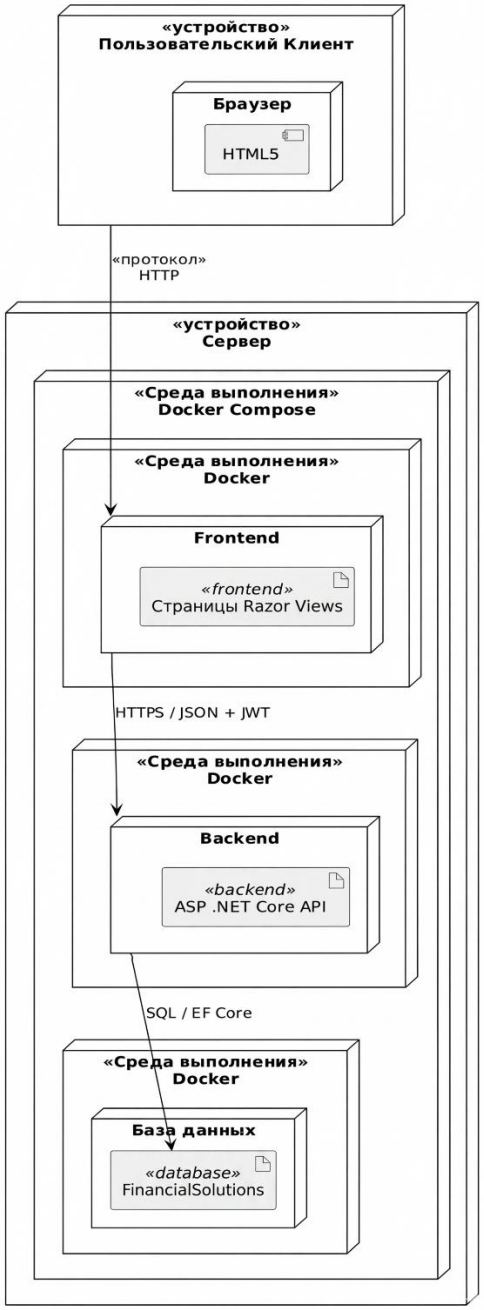

Рисунок 3 – Диаграмма развёртывания

2.3 Проектирование базы данных

Требуется разработать БД для хранения и управления данными. БД должна хранить в себе информацию о заявках и клиентах. На рисунке 4 в виде ERD показана физическая модель БД, спроектированная с использованием SSMS.

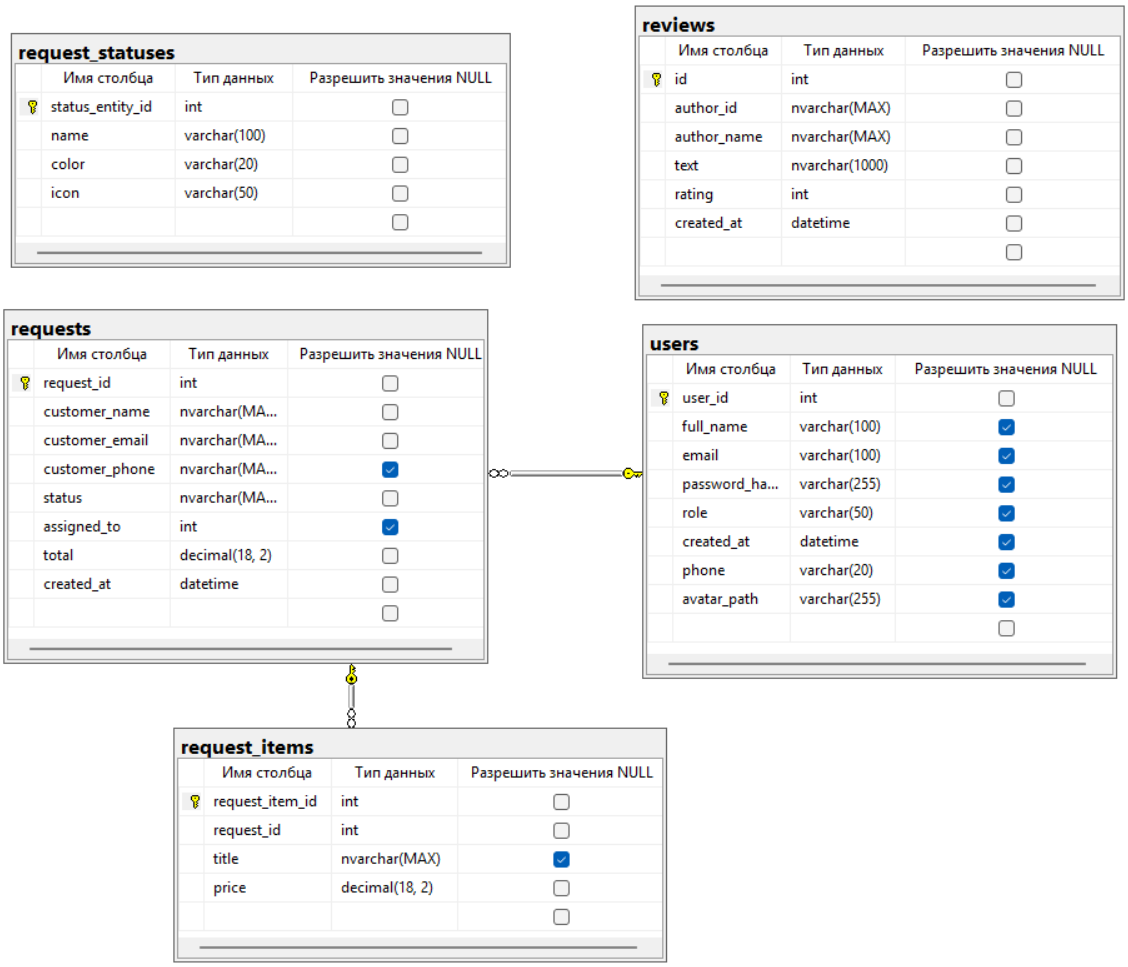

Рисунок 4 – SSMS. Физическая модель БД

3 Разработка и интеграция модулей программного обеспечения

3.1 Разработка программных модулей

В рамках курсового проекта разработан веб-сайт, включающий серверную часть и клиентский интерфейс на ASP.NET Core, разработанные в Visual Studio [2]. Для взаимодействия с БД использован EF Core и создан набор моделей и DTO-классов.

Для реализации получения заявок с сервера использован класс HttpClient. Фрагмент кода метода отправки заявок представлен листингом 1.

Листинг 1 – Фрагмент кода для отправки Post-запроса на сервер

```
     // API: отправка заявки (только авторизованные)
     [HttpPost]
     [Route("api/requests/submit")]
     [Authorize(AuthenticationSchemes =
wtBearerDefaults.AuthenticationScheme + "," +
CookieAuthenticationDefaults.AuthenticationScheme)]
     public async Task<IActionResult> Submit([FromBody]
RequestSubmitDto dto)
     {
         // Базовая проверка входных данных
         if (dto == null || dto.Items == null ||
!dto.Items.Any())
         {
             _logger.LogWarning("Submit called with empty dto by
{User}", User?.Identity?.Name ?? "unknown");
             return BadRequest(new { success = false, message =
"Пустая заявка" });
         }

         // Получение email
         var userEmailClaim =
User.FindFirst(System.Security.Claims.ClaimTypes.Email)?.Value
                              ?? User.FindFirst("email")?.Value;

         _logger.LogInformation("Submit called by {email} with
{count} items", userEmailClaim ?? "(no-email)",
dto.Items.Count);
```

```
        // Если нет email — не авторизован
        if (string.IsNullOrEmpty(userEmailClaim))
        {
            return Unauthorized(new { success = false, message =
"Не удалось определить пользователя" });
        };
```

Реализация метода загрузки заявок в ASP.NET Core [4] представлен листингом 2.

Листинг 2 – Фрагмент кода метода отображения заявок

```
     // Обработчик формы отправки заявки
     document.getElementById('requestForm')?.addEventListener('su
bmit', async function (e) {
        e.preventDefault();

        // Проверка, что клиент вошёл в систему
        if (!isAuthenticatedClient()) {
            alert('Требуется авторизация. Пожалуйста,
войдите.');

bootstrap.Modal.getOrCreateInstance(document.getElementById('aut
hModal')).show();
            return;
        }

        const items = getCart();
        if (!items.length) {
            alert('Корзина пуста');
            return;
        }

        const dto = {
            customerName:
document.getElementById('customerName').value.trim(),
            customerEmail:
document.getElementById('customerEmail').value.trim(),
            customerPhone:
document.getElementById('customerPhone').value.trim(),
            items: items.map(i => ({
                serviceId: parseInt(i.serviceId, 10),
                title: i.title,
                price: parseFloat(i.price)
            }))
        };

        try {
```

```
     // Отправка POST-запроса на API endpoint
     const fetchFn = (typeof fetchWithAuth === 'function') ?
fetchWithAuth : fetch;
             const res = await fetchFn('/api/requests/submit', {
                 method: 'POST',
                 headers: { 'Content-Type': 'application/json' },
                 body: JSON.stringify(dto)
             });

             // Чтение ответов
             let data = null;
             try {
                 const ct = res.headers.get('content-type') ||
'';
                 if (ct.includes('application/json')) {
                     data = await res.json();
                 } else {
                     data = { __rawText: await res.text() };
                 }
             } catch (parseErr) {
                 console.error('Не удалось распарсить тело
ответа', parseErr);
             };
```

3.2 Реализация интерфейса пользователя

Для отображения информации о заявках разработана страница Requests [4], фрагмент кода представлен листингом 3.

Листинг 3 – Фрагмент кода страницы Requests

```
// код отображения страницы управления заявками

     @section Scripts {
         <script>
             /*** Функция загрузки списка заявок и отрисовки
таблицы */
             async function loadRequests() {
                 // Выбор функции запроса (с авторизацией или
стандартный fetch)
       const fetchFn = (typeof fetchWithAuth === 'function') ?
fetchWithAuth : fetch;

       // Загрузка статусов
         let defs = [
            { name: 'Новая' },
            { name: 'В обработке' },
```

```
            { name: 'Завершена' },
            { name: 'Отменена' }];

                try {
                    const sres = await
fetchFn('/api/requests/statuses/definitions');
                    if (sres.ok) defs = await sres.json();
                } catch (e) {
                    console.warn('Не удалось загрузить
определения статусов', e);
                }

                // Загрузка основного списка заявок
                const res = await fetchFn('/api/requests');
                const data = await res.json();
                const container =
document.getElementById('requestsContainer');

                // Проверка на наличие данных
                if (!data || !data.length) {
                    container.innerHTML = '<p class="text-
muted">Заявок нет</p>';
                    return;
                }

                html += '</tbody></table>';
                container.innerHTML = html;

             // Запуск инициализации при загрузке страницы
             document.addEventListener('DOMContentLoaded',
loadRequests);
         </script>
     }
```

3.3 Разграничение прав доступа пользователей

В приложении разработано разграничение прав доступа пользователей с помощью JWT, а пароли надёжно хэшируются с помощью алгоритма HS256 [5]. Для этого в приложении реализована авторизация. Код авторизации с использованием JWT представлен листингом 4.

Листинг 4 – Код авторизации пользователей

```
// Контроллер аутентификации пользователей
 /// <summary>
        /// Авторизация пользователя
```

```
        /// POST: api/user/auth/login
        /// </summary>
        [HttpPost("login")]
        public async Task<IActionResult> Login([FromBody]
LoginRequest request)
        {
            try
            {
                if (string.IsNullOrWhiteSpace(request.Email) ||
string.IsNullOrWhiteSpace(request.Password))
                    return BadRequest(new { success = false,
message = "Email и пароль обязательны" });

                var token =
_authService.Authenticate(request.Email, request.Password);

                if (token == null)
                {
                    _logger.LogWarning($"Неудачная попытка входа
для {request.Email}");
                    return Unauthorized(new { success = false,
message = "Неверный email или пароль" });
                }

                var user = _context.Users.FirstOrDefault(u =>
u.Email == request.Email);

                if (user != null)
                {
     var claims = new List<Claim>
         {
           new Claim("id", user.UserId.ToString()),
           new Claim(ClaimTypes.Email, user.Email),
           new Claim(ClaimTypes.Name, user.FullName ?? ""),
           new Claim(ClaimTypes.Role, user.Role ?? "user")};
                    var identity = new ClaimsIdentity(claims,
CookieAuthenticationDefaults.AuthenticationScheme);
                    var principal = new
ClaimsPrincipal(identity);
                    await
HttpContext.SignInAsync(CookieAuthenticationDefaults.Authenticat
ionScheme, principal);
                }

                _logger.LogInformation($"Пользователь
{request.Email} успешно авторизован");

                return Ok(new
                {
                    success = true,
                    token,
                    expiresIn = 3600,
```

```
                    user = user == null ? null : new { id =
user.UserId, email = user.Email, fullName = user.FullName, role
= user.Role }
                });
            }
            catch (Exception ex)
            {
                _logger.LogError($"Ошибка при авторизации:
{ex.Message}");
                return StatusCode(500, new { success = false,
message = "Внутренняя ошибка сервера" });
            }
        }
```

3.4 Экспорт и импорт данных

Экспорт данных на веб-сайте реализован с помощью контроллера, код которого представлен листингом 5. Права доступа к экспорту есть только у пользователей из группы администраторы в целях безопасности.

Листинг 5 – Код метода экспорта данных

```
// Экспорт в формат CSV
[Authorize(AuthenticationSchemes =
JwtBearerDefaults.AuthenticationScheme + "," +
CookieAuthenticationDefaults.AuthenticationScheme, Roles =
"admin")]
public async Task<IActionResult> Export()
{
    // Функция экранирования двойных кавычек в CSV-ячейках
    static string Escape(string s) => (s ?? "").Replace("\"",
"\"\"");

    // Создание CSV-строки
    var sb = new StringBuilder();

sb.AppendLine("\"Id\",\"CustomerName\",\"CustomerEmail\",\"Custo
merPhone\",\"Items\",\"Total\",\"Status\",\"AssignedTo\",\"Creat
edAt\"");

    try
    {
        // Загрузка из БД всех заявок с элементами
        var rows = await _context.RequestEntities.Include(r =>
r.Items).OrderByDescending(r => r.CreatedAt).ToListAsync();
```

```
        foreach (var r in rows)
        {
            var itemsText = string.Join(" | ", (r.Items ?? new
List<RequestItemEntity>()).Select(i => $"{i.Title}
({i.Price})"));

            // Заполнение строк CSV с экранированием
            sb.AppendLine(

$"\"{Escape(r.RequestEntityId.ToString())}\",\"{Escape(r.Custome
rName)}\",\"{Escape(r.CustomerEmail)}\",\"{Escape(r.CustomerPhon
e ??
"")}\",\"{Escape(itemsText)}\",\"{Escape(r.Total.ToString())}\",
\"{Escape(r.Status)}\",\"{Escape(r.AssignedTo ??
"")}\",\"{Escape(r.CreatedAt.ToString(\"u\"))}\""
            );
        }
    }
    catch (Exception ex)
    {
        // Если чтение из БД не удалось, логируем и возвращаем
только заголовки
        _logger.LogWarning(ex, "Could not read RequestEntities
table for export — returning header-only CSV");
    }

    // Кодировка CSV в UTF-8
    var bytes = new UTF8Encoding(true).GetBytes(sb.ToString());

    // Возвращаем файл с содержимым CSV
    return File(bytes, "text/csv; charset=utf-8",
$"requests_{DateTime.UtcNow:yyyyMMdd_HHmmss}.csv");
}
```

4 Тестирование и отладка программного обеспечения

4.1 Структурное тестирование

Во время выполнения курсового проекта проведено структурное тестирование контролера AuthControllerTests [1] отвечающего за безопасность и разграничение прав доступа в системе. Фрагмент кода unit-теста на проверку корректности работы механизма аутентификации пользователей представлен листингом 6.

Листинг 6 – Фрагмент кода класса AuthControllerTests

```
// Код класса контроллера AuthControllerTests
public class AuthControllerTests
{
    [Fact]
    public async Task
Login_ReturnsUnauthorized_WhenCredentialsInvalid()
    {
        // Arrange — Настройка мок-объектов и окружения
        var authServiceMock = new Mock<AuthService>(null!,
null!);
        authServiceMock
            .Setup(s => s.Authenticate("wrong@example.com",
"badpass"))
            .Returns((string?)null);

        var loggerMock = new Mock<ILogger<AuthController>>();
        var options = new
DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase("testdb1").Options;
        using var context = new AppDbContext(options);

        var controller = new
AuthController(authServiceMock.Object, loggerMock.Object,
context);

        controller.ControllerContext = new ControllerContext
        {
            HttpContext = new DefaultHttpContext()
            {
                RequestServices = new ServiceCollection()
```

```
.AddSingleton(Mock.Of<IAuthenticationService>())
.BuildServiceProvider()
            }
        };

        var request = new LoginRequest { Email =
"wrong@example.com", Password = "badpass" };

        // Act — Выполнение тестируемого метода
        var result = await controller.Login(request);

        // Assert — Проверка результата (ожидаем ошибку доступа)
        Assert.IsType<UnauthorizedObjectResult>(result);
        var objResult = (UnauthorizedObjectResult)result;
        Assert.False(((dynamic)objResult.Value).success);
    }

    [Fact]
    public async Task Login_ReturnsOk_WhenCredentialsValid()
    {
        // Arrange — Настройка тестовых данных и базы в памяти
        var authServiceMock = new Mock<AuthService>(null!,
null!);
        authServiceMock
            .Setup(s => s.Authenticate("test@example.com",
"Secret123"))
            .Returns("test-jwt-token");

        var loggerMock = new Mock<ILogger<AuthController>>();
        var options = new
DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase("testdb2").Options;
        using var context = new AppDbContext(options);

        context.Users.Add(new User { UserId = 1, Email =
"test@example.com", FullName = "Test User", Role = "user",
PasswordHash = "..." });
        context.SaveChanges();

        var controller = new
AuthController(authServiceMock.Object, loggerMock.Object,
context);
        controller.ControllerContext = new ControllerContext
        {
            HttpContext = new DefaultHttpContext()
            {
                RequestServices = new ServiceCollection()

.AddSingleton(Mock.Of<IAuthenticationService>())
                    .BuildServiceProvider()
            }
        };
```

4.2 Функциональное тестирование

Во время проведения функционального тестирования проведена проверка веб-сайта методом «черного ящика», результаты тестирования представлены в таблице 1.

Таблица 1 – Набор тестов для приложения

Действие	Ожидаемый результат	Фактический результат
Нажать на кнопку «Вход»в верхнем углу экрана. Ввести в поле email «admin1@gmail.com» и пароль «wrongPassword»	Над кнопкой «Авторизоваться» отображается надпись «Неверный логин или пароль»	Совпадает с ожидаемым
Открыть страницу «Заявки». Нажать на статус заявки. Изменить статус заявки на «Готов»	Заявка отобразится со статусом со статусом «Готов»	Совпадает с ожидаемым
Войти под учётной записью Администратора. Нажать на кнопку «Админ» на панели управления. Нажать на кнопку «Экспорт Excel»	Таблица «Заявки» сохраниться в формате Excel-таблицы	Совпадает с ожидаемым
Нажать на кнопку «Профиль» на панели управления. Отредактировать имя на «Иван» и изменить фотографию в профиле	Корректное отображение изменённых данных в профиле пользователя	Совпадает с ожидаемым

На основе результатов тестирования можно сделать вывод: приложение работает корректно и согласно ожиданиям пользователя.

5 Инструкция по эксплуатации программного обеспечения

5.1 Установка программного обеспечения

Для развёртывания БД нужно подключиться к серверу MSSQL, с помощью SSMS.

Для установки серверной части требуется перейти в терминале в желаемою папку для API, склонировать репозиторий, создать файл appsettings.json по примеру файла appsettings.Development.json и в терминале использовать команду представленную листингом 7.

Листинг 7 – Код запуска API средствами docker-compose

```
# Код запуска API средствами docker-compose
docker-compose up -d –build
```

Для установки веб-сайта требуется распаковать courseProjAPI.zip и заменить appsettings.json с учётом нового сервера.

В приложении по умолчанию используются следующие учётные данные:

– логин: admin1@gmail.com;

– пароль: admin12.

5.2 Инструкция по работе

Для авторизации требуется ввести учётные данные в поля ввода email и пароля и нажать на кнопку «Авторизоваться». Окно авторизации показано на рисунке 5.

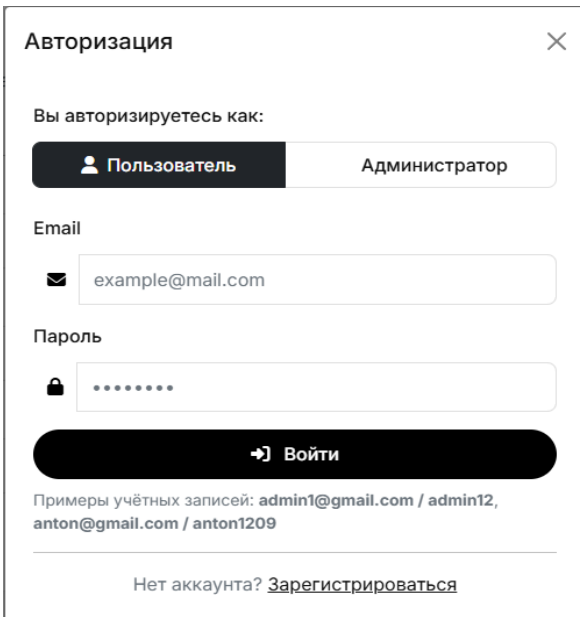

Рисунок 5 – Financial Solutions. Вид окна «Авторизация»

После авторизации пользователь с ролью «Администратор» перенаправляется на главное окно, где ему доступны страницы «Категории», «Услуги», «Заявки». Вид страницы «Услуги» представлен на рисунке 6.

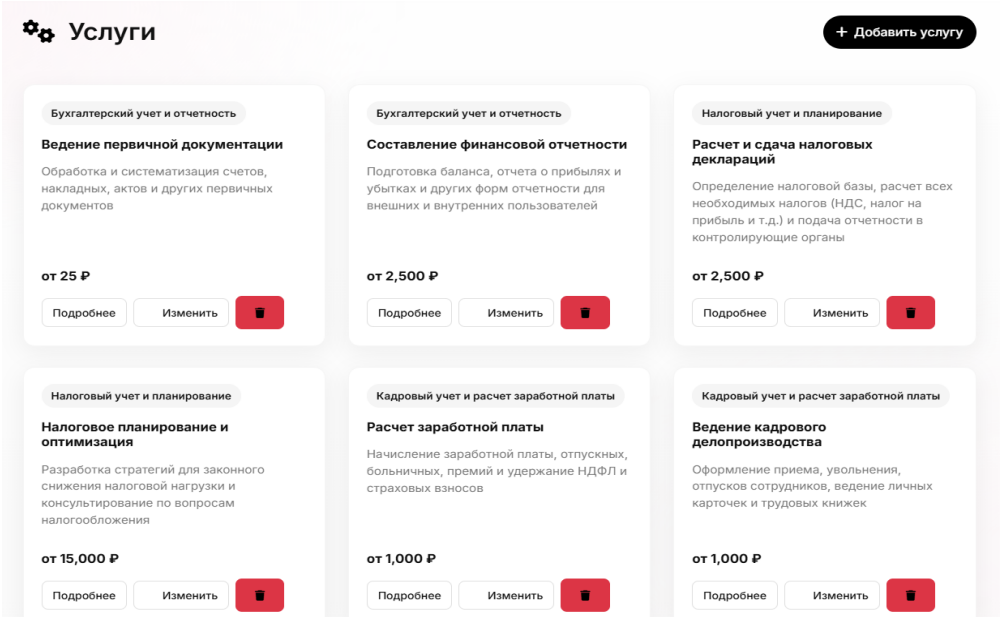

Рисунок 6 – Financial Solutions. Вид страницы «Услуги»

Для просмотра информации об услуге нужно нажать на кнопку «Подробнее», после этого отобразится окно с подробной информацией об услуге. Вид окна «Информация об услуге» представлен на рисунке 7.

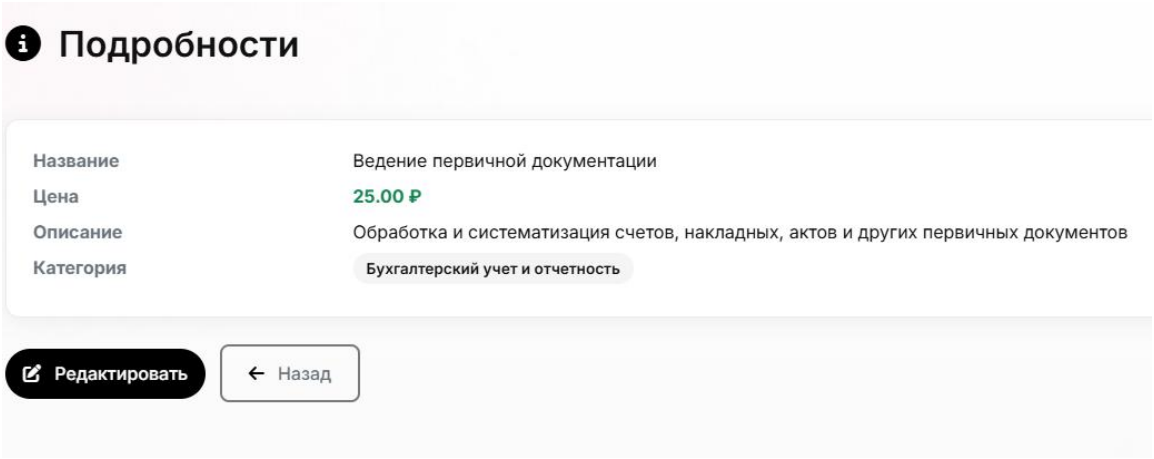

Рисунок 7 – Financial Solutions. Вид окна «Информация об услуге»

Для создания новой услуги нужно нажать кнопку «Добавить услугу», после этого отобразится форма для заполнения информации об услуге. Вид формы «Создание услуги» представлен на рисунке 8.

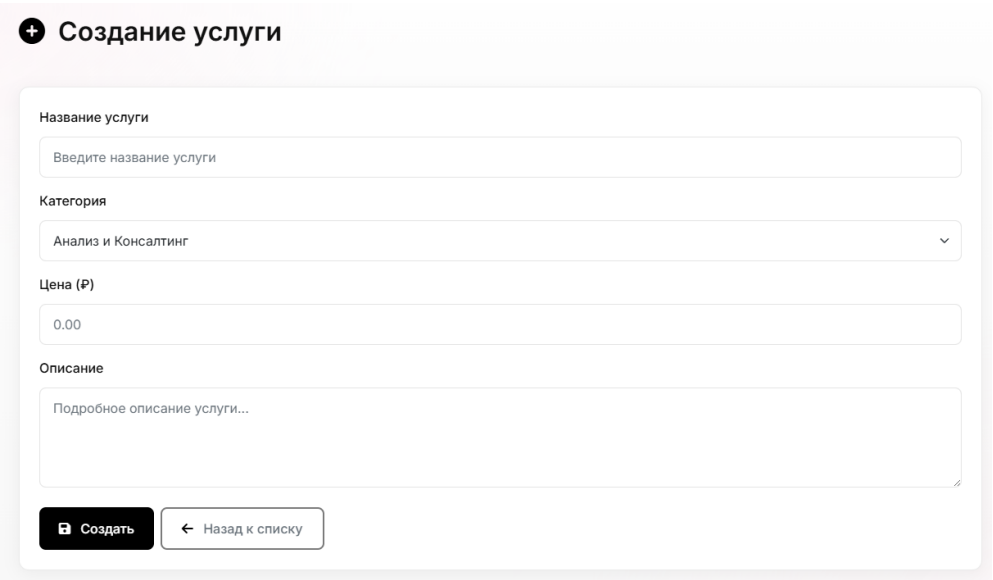

Рисунок 8 – Financial Solutions. Вид формы «Создание услуги»

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта достигнута поставленная цель: разработан веб-сайт для управления заявками на бухгалтерские услуги, обеспечивающее разграничение прав доступа пользователей, оформление и распределение заявок и отслеживание их статуса с использованием онлайн-доступа.

В ходе курсового проектирования решены все поставленные задачи:

- проведен сбор требований целевой аудитории;
- проанализированы информационные источники по предметной области;
- спроектирована архитектура веб-сайта;
- спроектирована диаграмма вариантов использования;
- выбран состав программных и технических средств для реализации веб-сайта;
- спроектирована БД;
- спроектирован интерфейс веб-сайта;
- создана БД в выбранной СУБД;
- разработан API для требуемых функций веб-сайта;
- реализовано разграничение прав доступа пользователей;
- разработан интерфейс пользователя;
- разработаны модули веб-сайта;
- реализована работа веб-сайта с сервером БД при помощи REST API;
- выполнено структурное и функциональное тестирование ПО;
- разработана программная и эксплуатационная документация.

В результате выполнения поставленных задач разработано приложение, отвечающее современным тенденциям разработки и требованиям заказчика.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бек, К. Экстремальное программирование: разработка через тестирование. – Санкт-Петербург : Питер, 2021. – 224 с. – URL: https://ibooks.ru/bookshelf/376974/reading (дата обращения: 12.11.2025). – Режим доступа: для зарегистрир. пользователей. – Текст: электронный.

2. Гагарина, Л. Г. Технология разработки программного обеспечения : учебное пособие / Л. Г. Гагарина, Е. В. Кокорева, Б. Д. Сидорова-Виснадул ; под ред. Л. Г. Гагариной. – Москва : ФОРУМ : ИНФРА-М, 2023. – 400 с. – URL: https://znanium.com/catalog/product/1895679 (дата обращения: 06.11.2025). – Режим доступа: по подписке. – Текст : электронный.

3. Мартишин, С. А. БД. Практическое применение СУБД SQL- и NoSQL-типа для проектирования информационных систем : учебное пособие / С. А. Мартишин, В. Л. Симонов, М. В. Храпченко. — Москва : ФОРУМ : ИНФРА-М, 2023. — 368 с. - URL: https://znanium.com/catalog/product/1912454 (дата обращения: 13.11.2025). – Режим доступа: по подписке. — Текст : электронный.

4. Тидвелл, Д. Разработка интерфейсов. Паттерны проектирования. 3-е изд. – Санкт-Петербург : Питер, 2022. – 560 с. – Текст : электронный. – URL: https://ibooks.ru/bookshelf/386796/reading (дата обращения: 24.11.2025). – Режим доступа: для зарегистрир. пользователей.

5. Федорова, Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности : учебное пособие. — Москва : КУРС : ИНФРА-М, 2024. — 336 с. – URL: https://znanium.ru/catalog/product/2083407 (дата обращения: 01.11.2025). – Режим доступа: по подписке. – Текст : электронный.